AIE1901: Large Language Models for Social Simulations

Assignment 1 — LLM Assisted Code Annotation

School of Artificial Intelligence, The Chinese University of Hong Kong, Shenzhen

Background

Welcome to your first assignment! In Lab 1, you learned how to set up and use Google Colab or Jupyter Notebook (creating notebooks, running cells, and saving/sharing) and how to register and use an LLM service. You also learned prompting best practices using the framework *Task* → *Context* → *References* → *Evaluate* → *Iterate*. This assignment asks you to apply those Lab 1 skills to produce clear annotations for a given Python module from a small Schelling-model codebase.

You are now explorers, stepping into a small simulation codebase inspired by Schelling's model. Each group will receive one mysterious Python file. Your mission is to uncover what this file is trying to do and explain it to others in plain English.

Think of your file as a single puzzle piece in a larger simulation. Don't worry if you don't see the full picture yet — just focus on your own piece, and make it as clear as possible for someone who has never coded before.

Note: *Lab 1 focused on tooling (Colab & LLM usage) rather than theory of Schelling. You're not required to run the simulation.*

Learning Objectives

By completing this assignment, you will be able to:

- Use Google Colab or Jupyter Notebook to open/edit Python files.
- Apply LLM prompting systematically to generate and refine code comments.
- Produce professional, accurate and readable annotations for peers.
- Document your LLM workflow (model used, prompts, iterations, and human verification).

Group Allocation

Each group annotates one module.

Group	File to Annotate
1	`initialization.py`
2	`agent_selector.py`
3	`target_selector.py`
4	`move_acceptor.py`
5	`stop_criterion.py`

Your Task

Each group is assigned one Python (.py) file from the Schelling-style simulation codebase. Your mission is to explore this file, explain it in plain English, and document your process with an LLM.

1. **File Overview:** An overview of this file: its purpose, main functions/logic, typical inputs/outputs, assumptions, and any uncertainties you can infer. (No need to run code.)
2. **Code Annotation with LLM Assistance:** Use an LLM to produce beginner-friendly annotations:
   - Add a clear **file-level header comment** summarizing what the file is for.
   - Write **function/class docstrings** describing purpose, arguments, returns, side effects, and assumptions.
   - Insert **inline comments** for non-obvious logic, key conditionals/loops, and data structures (explain the "why," not just restating code).
   - For **placeholders or incomplete parts**, clarify the intended role and expected inputs/outputs (do NOT implement missing features).

   When using the LLM, follow the Lab 1 prompt framework: **Task** → **Context** → **References** → **Evaluate & Iterate**. Keep your explanations clear, simple, and engaging for absolute beginners.
3. **Prompt Log:** Submit a short record of your "exploration journey": the model (e.g. DeepSeek) used, your prompts, and a brief note on what changed across iterations (if any).
4. **Handling Unknown Terms:** If you encounter unfamiliar terminology, discuss it within your group and/or consult a large language model. Record not only the discussion or prompts you used, but also a short reflection log (1–3 sentences) on what you learned from this process.

Deliverables

Submit a single ZIP named `AIE1901_A1_GroupX.zip` containing:

- **1. Annotated source file:** The assigned `.py` with your comments/docstrings added (Task 2).
- **2. LLM Usage & Reflection (.pdf or .doc/.docx), at least 200 words— include:**
  - Group number; names & student IDs.
  - File overview (Task 1).
  - Prompt log (Task 3).
  - Reflection log: what you learned from discussing unknown terms and consulting the LLM (Task 4).

Format & Style Requirements

- **Docstrings:** Use a simple and consistent style. Start with a short sentence explaining what the function or class does, followed by explanations of its inputs (arguments) and outputs (return values) in plain language.
- **Inline comments:** Use short, clear sentences above tricky lines or code blocks to explain what the code is doing and why. Avoid repeating the code in words (e.g., don't just say "increase count by 1" if the code is `count += 1`—explain the purpose instead).

- **English writing:** Keep it simple and beginner-friendly. Avoid programming jargon or explain it if you must use it.
- **No execution required:** You do not need to run the code. Some files may be incomplete. Focus on making the code easy to understand for a beginner who has never seen this file before.

Assessment Rubric (100 pts)

Criterion	Description	Pts
Accuracy	Comments/docstrings correctly describe behavior, parameters, and assumptions; no conceptual errors	35
Coverage & Depth	All functions/classes/critical blocks annotated; placeholders explained with intended roles	25
Clarity & Style	Readable, concise English; consistent docstring format; helpful inline comments	20
LLM Workflow Quality	Applied Task→Context→References→Evaluate→Iterate; documented prompts	15
Professionalism	File organization, naming, and submission formatting	5

Academic Integrity & LLM Use

- Collaboration is within group only.
- LLMs are allowed for drafting annotations, not for generating or modifying new logic.
- You must verify any LLM output; you are accountable for accuracy.
- Include your prompt log.

Submission

- **Where:** Blackboard
- **When:** Before 19 September 2025

Appendix: Sample Prompt (adapt/adopt in your workflow)

Annotate the following Python code for absolute beginners. Explain each line clearly, avoid technical jargon, and use friendly, easy-to-understand language. Add inline comments starting with # above or beside the relevant code. The annotated code will be used in an introductory programming class to help students understand both what the code does and why each step is necessary.

Reference:

```
# This is a simple program that prints Hello, World!
# print() is a function that displays text on the screen
print("Hello, World!")
```